

Interleaving String

Company: Microsoft

Year: 2025

Difficulty: Medium

Topic: Dynamic Programming, Strings

Source: [LeetCode 97 – Interleaving String](#)

Problem Description

You are given three strings s1, s2, and s3.

You need to determine whether s3 can be formed by **interleaving** s1 and s2.

While creating s3, we must maintain the original order of characters from both s1 and s2.

In other words:

- We can take the next character from s1, or
- We can take the next character from s2.
- We cannot change the order of characters within either string.

For example:

s1 = "ab"

s2 = "cd"

Possible interleavings include:

"acbd"

"acdb"

"cabd"

"cdab"

But:

"adbc"

is not possible because d comes before c from s2, changing the original order of s2.

Input Format

- The first line contains a string s1.
- The second line contains a string s2.
- The third line contains a string s3.

Output Format

- Print true if s3 can be formed by interleaving s1 and s2 while maintaining the relative order of characters in both strings.
- Otherwise, print false.

Constraints

- $0 \leq s1.length, s2.length \leq 100$
- $0 \leq s3.length \leq 200$
- s1, s2, and s3 consist of lowercase English letters.

Example 1

Input:

s1 = "aabcc"

s2 = "dbbca"

s3 = "aadbcbcbac"

Output:

true

Explanation

One possible way to form s3 is:

s1 = "aa" + "bc" + "c"

s2 = "dbbc" + "a"

Interleaving them:

"aa" + "dbbc" + "bc" + "a" + "c"

gives:

"aadbcbcbac"

Therefore, the answer is:

true

Example 2

Input:

s1 = "aabcc"

s2 = "dbbca"

s3 = "aadbbaacc"

Output:

false

Explanation

There is no valid way to choose characters from s1 and s2 while maintaining the original order of both strings and produce s3.

Therefore:

false

Example 3

Input:

s1 = ""

s2 = ""

s3 = ""

Output:

true

Explanation

All three strings are empty, so s3 is trivially an interleaving of s1 and s2.

Approach to Solve This Problem

This problem can be solved using **Dynamic Programming (DP)**.

The main difficulty is that at every character of s3, we have **two choices**:

- Take the next character from s1
- Take the next character from s2

We need to determine whether **any sequence of these choices** can successfully construct s3.

First check the lengths

For s3 to be an interleaving:

$$s3.length = s1.length + s2.length$$

If this condition is not satisfied, immediately return:

false

DP Definition

Let:

$dp[i][j]$

represent whether the first i characters of s1 and the first j characters of s2 can form the first i + j characters of s3.

For example:

$$dp[2][3] = \text{true}$$

means:

The first 2 characters of s1 and the first 3 characters of s2 can successfully form the first 5 characters of s3.

How Do We Calculate $dp[i][j]$?

At position:

$s3[i + j - 1]$

we have two possibilities.

Case 1: Take a character from s1

If:

$s1[i - 1] == s3[i + j - 1]$

then we can come from:

$dp[i - 1][j]$

So:

$dp[i][j] = dp[i][j] || dp[i - 1][j]$

Case 2: Take a character from s2

If:

$s2[j - 1] == s3[i + j - 1]$

then we can come from:

$dp[i][j - 1]$

So:

$dp[i][j] = dp[i][j] || dp[i][j - 1]$

Therefore, the transition is:

$dp[i][j] =$

$(dp[i - 1][j] \&\& s1[i - 1] == s3[i + j - 1])$

$||$

$(dp[i][j - 1] \&\& s2[j - 1] == s3[i + j - 1])$

Base Case

Before using any characters:

`dp[0][0] = true`

because two empty strings can form an empty string.

For example:

`s1 = "a"`

`s2 = "b"`

`s3 = "ab"`

Initially:

`dp[0][0] = true`

Then we can build the answer one character at a time.

Solution:

1. **Read the input:** `main()` reads the three strings `s1`, `s2`, and `s3`, matching the Input Format.
2. **Check the length condition:** If `s1.length() + s2.length()` is not equal to `s3.length()`, print false immediately because an interleaving must contain every character from both strings.
3. **Create the DP table:** Create a boolean table `dp` of size $(s1.length() + 1) \times (s2.length() + 1)$. `dp[i][j]` represents whether the first `i` characters of `s1` and the first `j` characters of `s2` can form the first `i + j` characters of `s3`.
4. **Initialize the base case:** Set `dp[0][0] = true` because two empty strings can form an empty string.
5. **Process characters from s1:** For each `i`, check whether `s1[i - 1]` matches the corresponding character in `s3`. If it matches and `dp[i - 1][j]` is true, then `dp[i][j]` can also be true.
6. **Process characters from s2:** Similarly, check whether `s2[j - 1]` matches the corresponding character in `s3`. If it matches and `dp[i][j - 1]` is true, then `dp[i][j]` can be true.
7. **Consider both choices:** At every position, a character can come from either `s1` or `s2`. Therefore, `dp[i][j]` is true if at least one of these two possibilities is valid.

8. **Print the result:** After filling the complete DP table, `dp[s1.length()][s2.length()]` tells us whether the entire `s3` can be formed. Print true if it is possible; otherwise, print false.

Java Solution

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String s1 = sc.nextLine();
        String s2 = sc.nextLine();
        String s3 = sc.nextLine();

        int m = s1.length();
        int n = s2.length();

        // Length must match
        if (m + n != s3.length()) {
            System.out.println(false);
            return;
        }
    }
}
```

```
boolean[][] dp = new boolean[m + 1][n + 1];
```

```
// Base case
```

```
dp[0][0] = true;
```

```
for (int i = 0; i <= m; i++) {
```

```
    for (int j = 0; j <= n; j++) {
```

```
        if (i == 0 && j == 0) {
```

```
            continue;
```

```
        }
```

```
        int k = i + j - 1;
```

```
        // Take character from s1
```

```
        if (i > 0 &&
```

```
            dp[i - 1][j] &&
```

```
            s1.charAt(i - 1) == s3.charAt(k)) {
```

```
                dp[i][j] = true;
```

```
        }
```

```
        // Take character from s2
```

```
        if (j > 0 &&
```

```
            dp[i][j - 1] &&
```



```
s2.charAt(j - 1) == s3.charAt(k)) {  
  
    dp[i][j] = true;  
}  
}  
}  
  
System.out.println(dp[m][n]);  
}  
}
```

Complexity Analysis

- **Time Complexity:** $O(m \times n)$ — every state in the DP table is calculated once.
- **Space Complexity:** $O(m \times n)$ — the DP table requires $(m + 1) \times (n + 1)$ boolean values.